

# What Tatami Guarantees

One theorem in four parts, and the 116 results they rest on. Every statement below is checked by Lean's kernel; none is assumed, and none is left open.

Checked by Lean 4.33.0, no Mathlib    Open goals none    Theorems 117  
docs/phase\_1/guarantees.html ·rebuild with make -C docs guarantees

01

## What “proved” means here

Every claim in this document is a theorem in the repository, accepted by Lean's type checker. Nothing is asserted in prose and taken on trust.

### No open goals

Lean records an unfinished proof as `sorry` and warns on every one. The build reports none. This is mechanical, not a claim about our diligence: `lake build` prints a warning for each, and there are no warnings to print.

### No extra axioms

`#print axioms pipeline_correct` reports `propext`, `Classical.choice` and `Quot.sound` — the three Lean itself is built on. In particular *not* `sorryAx`, which is what an unfinished proof would leave behind even if the warning were suppressed.

Mathlib is not used. Everything below rests on Lean's own library, which means the lemmas about lists, orders and sorting that the proofs need were either found in core or proved here.

02

## The guarantee

For a corpus inference accepted and a schema the generator accepted, four things hold of the output. This is one theorem, `pipeline_correct`; everything else in this document is what it rests on.

| # | GUARANTEE                                                                                                                                                                                                                                                                                                                                                | GROUPS                      |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| 1 | <p><a href="#">Well-formedness</a>. The emitted signature repeats no module name, no value name within a module, and no field name within a record. That is what “a legal OCaml signature” amounts to for this fragment.</p> <p>Without it the generator could emit two record fields of one name, which OCaml rejects outright.</p>                     | <code>gen_wellFormed</code> |
| 2 | <p><a href="#">Canonicity</a>. The schema is determined by the corpus as a collection of documents, not by the order they arrived in. Shuffle the corpus and the answer is <i>equal</i>, not merely equivalent.</p> <p>Without it two runs over the same data could disagree, and the output would depend on the order files happened to be read in.</p> | <code>infer_perm</code>     |

- 3 **Structure preservation.** The tree the documents induce and the graph a reader finds in the generated signatures are the same graph — every edge one has the other has, distinct tables get distinct modules, and every table is reachable from the root.

Each clause earns its place. Without the first two an edge could be lost; without the next two one could be invented; without injectivity two tables could collapse into one module and the agreement would be vacuous; without rootedness a table could be declared and never reached, which is a table the loader creates and never fills.

`gen_tree_complete`  
`gen_tree_sound`  
`gen_moduleName_injOn`  
`gen_rooted`

- 4 **Principality.** Each member is given the principal type of the values seen there: one that admits every one of them (*adequacy*), and the least type that does (*minimality*).

Adequacy alone is nearly free — a corpus of integers is adequately described by `float`, or by `int option`. Minimality rules the loose answers out, and the two together pin the type from both sides.

`infer_admits` `infer_least`

**What it does not say.** Every part is conditional on both stages returning `ok`. Both refuse inputs — a member holding two types with no common type, a schema whose tree a signature cannot express — and the guarantee is about what is emitted, not about what is accepted. It also says nothing about rows: the program emits a description of tables, and the loader that fills them is OCaml, outside what Lean sees here.

03

## What rests on what

The dependency tree, read off the proof terms themselves rather than written by hand: a result appears under another exactly when the second proof mentions the first. A result used in more than one place is expanded the first time it appears and marked thereafter.

### 1. Well-formedness

`signature_wellFormed` `Guarantees` The emitted signature repeats no module name, no value name within a module and no field name within a record.

└ `gen_wellFormed` `Wellformed` Every generated signature is well formed: no module, value or record field name repeats.  
 └ `certify_wellFormed` `Wellformed` The generator checks its own output before returning it.  
 └ `okB_sound` `Wellformed` That check means what the proposition says.  
 └ `nodupNames_sound` `Wellformed` And so does its distinctness test.

## 2. Canonicity

`schema_canonical` Guarantees The schema is determined by the corpus as a collection of documents, not by the order they arrived in -- shuffle it and the answer is equal, not merely equivalent.

`infer_perm` Inference The schema is a property of the corpus and not of the order its documents arrived in: shuffle the corpus and you get the same answer.

`Tables.merge_ok` Merge Merging preserves the order invariant.

`TableObs.merge_ok` Merge The same, one level down.

`Obs.merge_ok` Merge And one level down again.

`tyOrder` Order So are types.

`unionBy_sorted` Sorted And preserves order.

`belowK_of_lt` Sorted Transitivity carries `below` down from a smaller key.

`unionBy_belowK` Sorted A key below both sets is below their union.

`Tables.merge_right_comm` Merge Together: two documents merged into an accumulator commute, which is what the corpus fold needs.

`Tables.merge_assoc` Merge And grouping three of them either way gives the same answer.

`TableObs.merge_assoc` Merge The same, one level down.

`Obs.merge_assoc` Merge And one level down again.

`tyOrder` Order *shown above*

`unionBy_assoc` Sorted And associative.

`sortedK_ext` Sorted Two sorted sets with the same members are the same list.

`belowK_of_lt` Sorted *shown above*

`memBy_of_below` Sorted A key below everything in a set is not in it.

`unionBy_mem` Sorted A union is exactly the two memberships, disjointed.

`belowK_of_lt` Sorted *shown above*

`memBy_of_below` Sorted *shown above*

`unionBy_sorted` Sorted *shown above*

`Obs.merge_ok` Merge *shown above*

`mergeBy_assocOn` Sorted Merging is associative, for the same reason.

`lookupBy_allV` Sorted A value looked up satisfies whatever the list's values do.

`mergeBy_lookup` Sorted A merge is exactly its two lookups combined -- this is the whole content of merging, and both laws below read it off.

`below_of_lt` Sorted The same, for association lists.

`lookupBy_of_below` Sorted A key below everything in a list is not in it.

`mergeBy_sorted` Sorted Merging two ordered lists gives an ordered list.

`below_of_lt` Sorted *shown above*

`mergeBy_below` Sorted A key below both lists is below their merge.

`sorted_ext` Sorted Two sorted lists that agree on every lookup are the same list.

`below_of_lt` Sorted *shown above*

`lookupBy_of_below` Sorted *shown above*

`strOrder` Order Member names are strictly totally ordered.

`TableObs.merge_ok` Merge *shown above*

`pathOrder` Order So are table paths.

`Tables.merge_comm` Merge Merging two documents' observations gives the same answer in either order.

|                     |           |                                                                                                                      |
|---------------------|-----------|----------------------------------------------------------------------------------------------------------------------|
| TableObs.merge_comm | Merge     | The same, one level down, for a single table's observations.                                                         |
| Obs.merge_comm      | Merge     | And one level down again, for a single member's: counts add, type sets union, flags disjoint.                        |
| tyOrder             | Order     | shown above                                                                                                          |
| unionBy_comm        | Sorted    | The same for a set: union is commutative.                                                                            |
| mergeBy_commOn      | Sorted    | Merging is commutative, because combining lookups is.                                                                |
| lookupBy_allV       | Sorted    | shown above                                                                                                          |
| mergeBy_lookup      | Sorted    | shown above                                                                                                          |
| mergeBy_sorted      | Sorted    | shown above                                                                                                          |
| sorted_ext          | Sorted    | shown above                                                                                                          |
| strOrder            | Order     | shown above                                                                                                          |
| pathOrder           | Order     | shown above                                                                                                          |
| TablesOk.nil        | Merge     | Observing nothing is trivially in order.                                                                             |
| except_bind_ok      | Sorted    | A computation that succeeded had both halves succeed.                                                                |
| foldl_perm_on       | Inference | The same, where the laws hold only of values satisfying an invariant.                                                |
| mapM_perm           | Inference | Permuted inputs give permuted outputs, when both succeed.                                                            |
| except_bind_ok      | Sorted    | shown above                                                                                                          |
| mapM_ok_of_all      | Inference | If every element succeeds then the whole list does.                                                                  |
| mapM_ok_pointwise   | Sorted    | And every input reached some output.                                                                                 |
| except_bind_ok      | Sorted    | shown above                                                                                                          |
| mem_of_mapM_ok      | Sorted    | Every output of a mapped list came from some input.                                                                  |
| except_bind_ok      | Sorted    | shown above                                                                                                          |
| observeDocument_ok  | Walk      | So one document, observed from nothing, comes out in order.                                                          |
| TablesOk.nil        | Merge     | shown above                                                                                                          |
| observe_ok          | Walk      | Everything the inference walk records is kept in key order -- tables by path, members by name, types by their order. |
| Tables.upsert_ok    | Merge     | Recording a table keeps everything in order.                                                                         |
| TableObsOk.empty    | Merge     | So is a table with no members yet.                                                                                   |
| allV_map            | Sorted    | And preserves a property of the values.                                                                              |
| insertBy_allV       | Sorted    | And preserves a property of the values.                                                                              |
| insertBy_sorted     | Sorted    | Inserting one entry keeps the list ordered.                                                                          |
| below_of_lt         | Sorted    | shown above                                                                                                          |
| insertBy_below      | Sorted    | And keeps a smaller key below it.                                                                                    |
| pathOrder           | Order     | shown above                                                                                                          |
| sortedBy_map        | Sorted    | Rewriting values in place leaves the order alone.                                                                    |
| below_map           | Sorted    | So it leaves `below` alone.                                                                                          |
| except_bind_ok      | Sorted    | shown above                                                                                                          |
| keep                | Walk      | Each record update the walk makes leaves the member list alone.                                                      |
| recordMember_ok     | Merge     | So does recording one observation at one member.                                                                     |
| Obs.merge_ok        | Merge     | shown above                                                                                                          |
| Tables.upsert_ok    | Merge     | shown above                                                                                                          |
| insertBy_allV       | Sorted    | shown above                                                                                                          |
| insertBy_sorted     | Sorted    | shown above                                                                                                          |
| strOrder            | Order     | shown above                                                                                                          |



### 3. Structure preservation

**structure\_preserved** Guarantees The tree the documents induce and the graph a reader finds in the generated signatures are the same graph, and it is rooted.

- **gen\_moduleName\_injOn** Tree Two different tables never get the same module name, so the file's graph is the same graph and not a collapse of it.

- **genModule\_name** Tree A table's module is named after that table.

- **except\_bind\_ok** Sorted *shown above*

- **gen\_faithful** Tree The generator checks its input has the shape a corpus produces, so the tree properties hold of whatever it is handed.

- **gen\_schemaOk** Tree If generation succeeded, the schema passed that check.

- **nodupNames\_sound** Wellformed *shown above*

- **nodupPaths\_sound** Tree The check that no two tables share a path means what it says.

- **path\_mem** Tree A path the check found in the schema is the path of a table of it.

- **gen\_modules** Tree Every module in the output came from one of the three generation passes.

- **except\_bind\_ok** Sorted *shown above*

- **gen\_inv** Tree Generation is the raw pass followed by a check that hands back its input unchanged.

- **noClash\_modules** Tree The duplicate-name check hands back its input unchanged.

- **gen\_wellFormed** Wellformed *shown above*

- **inj\_of\_nodup\_map** Tree If a list's keys are all distinct then two elements with the same key are the same element.

- **mapM\_ok\_map** Sorted A mapped list agrees pointwise with its inputs under any function that agrees on each.

- **except\_bind\_ok** Sorted *shown above*

- **mem\_orderTables\_complete** Tree The same, said of the sort as a whole.

- **mem\_orderGo\_complete** Tree And it loses none, given that no two tables share a path.

- **inj\_of\_nodup\_map** Tree *shown above*

- **gen\_rooted** Tree Every table is reachable from the root, so walking the signatures from the root module enumerates the whole schema.

- **gen\_schemaOk** Tree *shown above*

- **reachable\_sound** Tree Everything the search reports is genuinely reachable from the root.

- **reachGo\_sound** Tree So does the whole search.

- **stepReach\_sound** Tree One round of the reachability search only ever reports tables genuinely one edge further out.

- **gen\_tree\_complete** Tree Nothing in the document's tree is lost: every nesting the documents induce is declared in the generated signatures.

- **gen\_tree\_complete\_coll** Tree The half of that for collections: a table whose rows sit inside another declares the lookup that finds them.

- **genModule\_lookup** Tree A table whose rows sit in a collection declares the lookup that finds them from the holder's key.

- **except\_bind\_ok** Sorted *shown above*

- **genModule\_name** Tree *shown above*

- **gen\_faithful** Tree *shown above*

- **gen\_module\_of** Tree Every table of the schema has a module in the output.

- **gen\_faithful** Tree *shown above*

- **gen\_modules** Tree *shown above*

- **mapM\_ok\_pointwise** Sorted *shown above*

|                         |                          |                                                                                                                                           |                                                                                              |
|-------------------------|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
|                         | mem_orderTables_complete | Tree                                                                                                                                      | shown above                                                                                  |
| gen_tree_complete_ref   | Tree                     | The half for references: a member that was an object and became a table leaves an accessor holding that table's key.                      |                                                                                              |
| genModule_accessor      | Tree                     | Every column of the layout becomes an accessor in the signature.                                                                          |                                                                                              |
|                         | except_bind_ok           | Sorted                                                                                                                                    | shown above                                                                                  |
| genModule_layout        | Tree                     | A module was only generated if its column layout was computed first.                                                                      |                                                                                              |
|                         | except_bind_ok           | Sorted                                                                                                                                    | shown above                                                                                  |
| genModule_lookup_only   | Tree                     | And that is the only declaration of that shape, so a reader who finds one has found a parent and not something that merely resembles one. |                                                                                              |
|                         | except_bind_ok           | Sorted                                                                                                                                    | shown above                                                                                  |
| genModule_name          | Tree                     | shown above                                                                                                                               |                                                                                              |
| gen_faithful            | Tree                     | shown above                                                                                                                               |                                                                                              |
| gen_moduleName_injOn    | Tree                     | shown above                                                                                                                               |                                                                                              |
| gen_module_of           | Tree                     | shown above                                                                                                                               |                                                                                              |
| gen_module_source       | Tree                     | Every module carrying any declaration at all is some table's.                                                                             |                                                                                              |
|                         | genLoader_decls          | Tree                                                                                                                                      | A loader has no signature at all, so it carries no edges.                                    |
|                         |                          | except_bind_ok                                                                                                                            | Sorted shown above                                                                           |
|                         | genRegistry_decls        | Tree                                                                                                                                      | Neither does the registry.                                                                   |
|                         | gen_modules              | Tree                                                                                                                                      | shown above                                                                                  |
|                         | mem_of_mapM_ok           | Sorted                                                                                                                                    | shown above                                                                                  |
|                         | mem_orderTables_sound    | Tree                                                                                                                                      | The same, said of the sort as a whole.                                                       |
|                         |                          | mem_orderGo_sound                                                                                                                         | Tree The compile-order sort invents no tables.                                               |
|                         | inj_of_nodup_map         | Tree                                                                                                                                      | shown above                                                                                  |
| keyTyOf_id_not_ref      | Tree                     | A member called `id` that is an object is refused outright, so a reference never hides in the key.                                        |                                                                                              |
|                         | find?_id_eq              | Tree                                                                                                                                      | With distinct member names, looking up the key finds the column that is there to be found.   |
|                         |                          | inj_of_nodup_map                                                                                                                          | Tree shown above                                                                             |
| layoutOf_keyTy          | Tree                     | If the layout was computed, the key's type was settled first.                                                                             |                                                                                              |
|                         | except_bind_ok           | Sorted                                                                                                                                    | shown above                                                                                  |
| layoutOf_mapM           | Tree                     | If the layout was computed, every member was considered.                                                                                  |                                                                                              |
|                         | except_bind_ok           | Sorted                                                                                                                                    | shown above                                                                                  |
| layoutOf_member         | Tree                     | A column produced for a member of the table reaches the layout.                                                                           |                                                                                              |
|                         | except_bind_ok           | Sorted                                                                                                                                    | shown above                                                                                  |
|                         | mapM_ok_pointwise        | Sorted                                                                                                                                    | shown above                                                                                  |
|                         | mapM_ok_pointwise        | Sorted                                                                                                                                    | shown above                                                                                  |
|                         | memberCol_ref            | Tree                                                                                                                                      | A member that is an object and is not the key becomes a column holding that object's key.    |
| gen_tree_sound          | Tree                     | Nothing is invented: every edge a reader finds between two tables is one the documents actually induced.                                  |                                                                                              |
|                         | baseCols_ref             | Tree                                                                                                                                      | Of the columns a table's position adds, only the back reference is a key into another table. |
|                         |                          | keyTyOf_not_ref                                                                                                                           | Tree A table's key is never an object.                                                       |
| colTyExpr_ref           | Tree                     | Only a reference column is typed as another module's key; a collection becomes a list and a scalar a scalar.                              |                                                                                              |
| genModule_accessor_only | Tree                     | And accessors are the only declarations that take a row and return another module's key.                                                  |                                                                                              |

- `except_bind_ok` `Sorted` *shown above*
- `genModule_lookup` `Tree` *shown above*
- `genModule_lookup_only` `Tree` *shown above*
- `genModule_name` `Tree` *shown above*
- `gen_faithful` `Tree` *shown above*
- `gen_moduleName_injOn` `Tree` *shown above*
- `gen_module_source` `Tree` *shown above*
- `inj_of_nodup_map` `Tree` *shown above*
- `mem_layoutOf` `Tree` Every member that is neither the table's key nor a collection becomes a column, under its mangled name, at the type inference gave it.
- `except_bind_ok` `Sorted` *shown above*
- `mem_of_mapM_ok` `Sorted` *shown above*
- `memberCol_ty` `Tree` A member column keeps the member's own type.

## 4. Principality

`types_principal` `Guarantees` Each member is given the principal type of the values seen there: one that admits every one of them, and the least type that does.

- `infer_admits` `Inference` Every type ever seen at a member fits the type that member was given.
  - `obs_admits` `Inference` Said of one member's observations on their own.
    - `foldJoin_le` `Inference` Folding the type-join over a list of observed types lands above every one of them.
      - `foldJoin_none` `Inference` Once the join is undefined it stays undefined, so a fold that succeeded never passed through failure.
      - `join_idem` `Lattice` A type joined with itself is itself.
      - `le_join_left` `Lattice` The common type is above the first argument.
      - `le_join_right` `Lattice` And above the second.
      - `join_comm` `Lattice` The common type of two types does not depend on which is named first.
      - `le_join_left` `Lattice` *shown above*
      - `le_trans` `Inference` The type order is transitive.
        - `join_assoc` `Lattice` Nor on how three of them are grouped, which is what makes folding observations order-independent.
  - `infer_least` `Inference` And that type is the least one that fits, so nothing is widened further than the data forces.
    - `obs_least` `Inference` Said of one member's observations on their own.
      - `bot_le` `Lattice` The empty type sits below everything.
      - `foldJoin_least` `Inference` And lands below anything else that is above them all.
        - `foldJoin_none` `Inference` *shown above*
        - `join_least` `Lattice` The common type is the \*least\* type above both -- the tightest answer, not merely an answer.

## Every property, in full

All 117 results, grouped by the area they belong to: what each says in plain words, and the Lean statement verbatim.

## Guarantees 5

The summary theorem itself.

### pipeline\_correct

Everything below in one statement: for a corpus inference accepted and a schema the generator accepted, the signature is well formed, the schema is canonical, the structure is preserved and the types are principal.

```
theorem pipeline_correct (cfg : Config) (nm : Naming) (ds : List Doc)
  (ts : Tables) (f : File)
  (hinf : inferCorpus cfg ds = .ok ts)
  (hgen : gen nm (toSchema ts) = .ok f) :
  f.WellFormed
  ∧ (∀ es ts', ds.Perm es → inferCorpus cfg es = .ok ts' → ts = ts')
  ∧ ((∀ p q, CollEdge (toSchema ts) p q →
    FileCollEdge f (moduleName nm p) (moduleName nm q))
    ∧ (∀ p q, RefEdge (toSchema ts) p q →
    FileRefEdge f (moduleName nm p) (moduleName nm q))
    ∧ (∀ t u, t ∈ toSchema ts → u ∈ toSchema ts →
    FileCollEdge f (moduleName nm t.path) (moduleName nm u.path) →
    CollEdge (toSchema ts) t.path u.path)
    ∧ (∀ t u, t ∈ toSchema ts → u ∈ toSchema ts →
    FileRefEdge f (moduleName nm t.path) (moduleName nm u.path) →
    RefEdge (toSchema ts) t.path u.path)
    ∧ (∀ t u, t ∈ toSchema ts → u ∈ toSchema ts →
    moduleName nm t.path = moduleName nm u.path → t.path = u.path)
    ∧ Rooted (toSchema ts))
  ∧ (∀ p t, (p, t) ∈ ts → ∀ k o, (k, o) ∈ t.members → ∀ τ, o.joined = some τ →
    (∀ ty, ty ∈ o.seen → ty ⊆ τ) ∧ (∀ σ, (∀ ty, ty ∈ o.seen → ty ⊆ σ) → τ ⊆ σ))
```

### schema\_canonical

The schema is determined by the corpus as a collection of documents, not by the order they arrived in -- shuffle it and the answer is equal, not merely equivalent.

```
theorem schema_canonical (cfg : Config) (ds : List Doc) (ts : Tables)
  (hinf : inferCorpus cfg ds = .ok ts) :
  ∀ es ts', ds.Perm es → inferCorpus cfg es = .ok ts' → ts = ts'
```

### signature\_wellFormed

The emitted signature repeats no module name, no value name within a module and no field name within a record.

```
theorem signature_wellFormed (nm : Naming) (s : Schema) (f : File)
  (hgen : gen nm s = .ok f) : f.WellFormed
```

---

## structure\_preserved

The tree the documents induce and the graph a reader finds in the generated signatures are the same graph, and it is rooted.

```
theorem structure_preserved (nm : Naming) (s : Schema) (f : File)
  (hgen : gen nm s = .ok f) :
  -- nothing lost
  (∀ p q, CollEdge s p q → FileCollEdge f (moduleName nm p) (moduleName nm q))
  ∧ (∀ p q, RefEdge s p q → FileRefEdge f (moduleName nm p) (moduleName nm q))
  -- nothing invented
  ∧ (∀ t u, t ∈ s → u ∈ s →
    FileCollEdge f (moduleName nm t.path) (moduleName nm u.path) →
    CollEdge s t.path u.path)
  ∧ (∀ t u, t ∈ s → u ∈ s →
    FileRefEdge f (moduleName nm t.path) (moduleName nm u.path) →
    RefEdge s t.path u.path)
  -- the same graph, not a quotient of it
  ∧ (∀ t u, t ∈ s → u ∈ s →
    moduleName nm t.path = moduleName nm u.path → t.path = u.path)
  -- and rooted
  ∧ Rooted s
```

## types\_principal

Each member is given the principal type of the values seen there: one that admits every one of them, and the least type that does.

```
theorem types_principal (cfg : Config) (ds : List Doc) (ts : Tables)
  (hinf : inferCorpus cfg ds = .ok ts) :
  ∀ p t, (p, t) ∈ ts → ∀ k o, (k, o) ∈ t.members → ∀ τ, o.joined = some τ →
  (∀ ty, ty ∈ o.seen → ty ⊆ τ) ∧ (∀ σ, (∀ ty, ty ∈ o.seen → ty ⊆ σ) → τ ⊆ σ)
```

## Tree <sup>42</sup>

That the nesting of the JSON and the shape of the generated signatures are the same tree. The largest group, because the correspondence has to be proved in both directions and the generator taken apart to do it.

---

## baseCols\_ref

Of the columns a table's position adds, only the back reference is a key into another table.

```
theorem baseCols_ref {nm : Naming} {t : Table} {kt : Ty} {c : Col} {p : Path}
  (hkt : keyTyOf t = .ok kt) (hc : c ∈ baseCols nm t kt) (hty : c.ty = .ref p) :
  t.parent = some p
```

## colTyExpr\_ref

Only a reference column is typed as another module's key; a collection becomes a list and a scalar a scalar.

```
theorem colTyExpr_ref {nm : Naming} {c : Col} {a : String}
  (h : colTyExpr nm c = TyExpr.qualified a "id" v
    colTyExpr nm c = (TyExpr.qualified a "id").option) :
  ∃ p, c.ty = .ref p ∧ a = moduleName nm p
```

## find?\_id\_eq

With distinct member names, looking up the key finds the column that is there to be found.

```
theorem find?_id_eq : ∀ {l : List Column}, (l.map Column.name).Nodup →  
  ∀ {c : Column}, c ∈ l → c.name = "id" →  
  l.find? (fun x => x.name == "id") = some c
```

## genLoader\_decls

A loader has no signature at all, so it carries no edges.

```
theorem genLoader_decls {s : Schema} {nm : Naming} {t : Table} {m : Module}  
  (h : genLoader s nm t = .ok m) : m.decls = []
```

## genModule\_accessor

Every column of the layout becomes an accessor in the signature.

```
theorem genModule_accessor {nm : Naming} {t : Table} {m : Module} {cols : List Col} {c : Col}  
  (h : genModule nm t = .ok m) (hl : layoutOf nm t = .ok cols) (hc : c ∈ cols) :  
  Decl.value c.name (.arrow (.named "t") (colTyExpr nm c)) ∈ m.decls
```

## genModule\_accessor\_only

And accessors are the only declarations that take a row and return another module's key.

```
theorem genModule_accessor_only {nm : Naming} {t : Table} {m : Module}  
  {n : String} {X : TyExpr}  
  (h : genModule nm t = .ok m)  
  (hd : Decl.value n ((TyExpr.named "t").arrow X) ∈ m.decls)  
  (hX : ∀ Y Z, X ≠ TyExpr.arrow Y Z) :  
  ∃ cols c, layoutOf nm t = .ok cols ∧ c ∈ cols ∧ X = colTyExpr nm c
```

## genModule\_layout

A module was only generated if its column layout was computed first.

```
theorem genModule_layout {nm : Naming} {t : Table} {m : Module}  
  (h : genModule nm t = .ok m) : ∃ cols, layoutOf nm t = .ok cols
```

## genModule\_lookup

A table whose rows sit in a collection declares the lookup that finds them from the holder's key.

```
theorem genModule_lookup {nm : Naming} {t : Table} {m : Module} {pp : Path}  
  (h : genModule nm t = .ok m) (hp : t.parent = some pp) :  
  Decl.value (childLookupName nm pp)  
    (.arrow (.qualified (moduleName nm pp) "id") (.list (.named "t")))) ∈ m.decls
```

## genModule\_lookup\_only

And that is the only declaration of that shape, so a reader who finds one has found a parent and not something that merely resembles one.

```
theorem genModule_lookup_only {nm : Naming} {t : Table} {m : Module} {a n : String}  
  (h : genModule nm t = .ok m)  
  (hd : Decl.value n ((TyExpr.qualified a "id").arrow (TyExpr.list (TyExpr.named "t")))  
    ∈ m.decls) :  
  ∃ pp, t.parent = some pp ∧ a = moduleName nm pp
```

### genModule\_name

A table's module is named after that table.

```
theorem genModule_name {nm : Naming} {t : Table} {m : Module}
  (h : genModule nm t = .ok m) : m.name = moduleName nm t.path
```

### genRegistry\_decls

Neither does the registry.

```
theorem genRegistry_decls {s : Schema} {nm : Naming} {m : Module}
  (h : genRegistry s nm = .ok m) : m.decls = []
```

### gen\_faithful

The generator checks its input has the shape a corpus produces, so the tree properties hold of whatever it is handed.

```
theorem gen_faithful (nm : Naming) (s : Schema) (f : File) (h : gen nm s = .ok f) :
  Faithful s
```

### gen\_inv

Generation is the raw pass followed by a check that hands back its input unchanged.

```
theorem gen_inv {nm : Naming} {s : Schema} {f : File} (h : gen nm s = .ok f) :
  genRaw nm s = .ok f
```

### gen\_moduleName\_injOn

Two different tables never get the same module name, so the file's graph is the same graph and not a collapse of it.

```
theorem gen_moduleName_injOn (nm : Naming) (s : Schema) (f : File)
  (h : gen nm s = .ok f) :
  ∀ t u, t ∈ s → u ∈ s → moduleName nm t.path = moduleName nm u.path → t.path = u.path
```

### gen\_module\_of

Every table of the schema has a module in the output.

```
theorem gen_module_of {nm : Naming} {s : Schema} {f : File} {t : Table}
  (h : gen nm s = .ok f) (ht : t ∈ s) :
  ∃ m, m ∈ f.modules ∧ genModule nm t = .ok m
```

### gen\_module\_source

Every module carrying any declaration at all is some table's.

```
theorem gen_module_source {nm : Naming} {s : Schema} {f : File} {m : Module}
  (h : gen nm s = .ok f) (hm : m ∈ f.modules) (hd : m.decls ≠ []) :
  ∃ t, t ∈ s ∧ genModule nm t = .ok m
```

## gen\_modules

Every module in the output came from one of the three generation passes.

```
theorem gen_modules {nm : Naming} {s : Schema} {f : File} (h : gen nm s = .ok f) :  
  ∃ units loaders registry,  
    (orderTables s).mapM (genModule nm) = .ok units ∧  
    (orderTables s).mapM (genLoader s nm) = .ok loaders ∧  
    genRegistry s nm = .ok registry ∧  
    f.modules = units ++ loaders ++ [registry]
```

## gen\_rooted

Every table is reachable from the root, so walking the signatures from the root module enumerates the whole schema.

```
theorem gen_rooted (nm : Naming) (s : Schema) (f : File) (h : gen nm s = .ok f) :  
  Rooted s
```

## gen\_schemaOk

If generation succeeded, the schema passed that check.

```
theorem gen_schemaOk {nm : Naming} {s : Schema} {f : File} (h : gen nm s = .ok f) :  
  schemaOkB s = true
```

## gen\_tree\_complete

Nothing in the document's tree is lost: every nesting the documents induce is declared in the generated signatures.

```
theorem gen_tree_complete (nm : Naming) (s : Schema) (f : File)  
  (h : gen nm s = .ok f) :  
    (∀ p q, CollEdge s p q → FileCollEdge f (moduleName nm p) (moduleName nm q)) ∧  
    (∀ p q, RefEdge s p q → FileRefEdge f (moduleName nm p) (moduleName nm q))
```

## gen\_tree\_complete\_coll

The half of that for collections: a table whose rows sit inside another declares the lookup that finds them.

```
theorem gen_tree_complete_coll (nm : Naming) (s : Schema) (f : File)  
  (h : gen nm s = .ok f) :  
    ∀ p q, CollEdge s p q → FileCollEdge f (moduleName nm p) (moduleName nm q)
```

## gen\_tree\_complete\_ref

The half for references: a member that was an object and became a table leaves an accessor holding that table's key.

```
theorem gen_tree_complete_ref (nm : Naming) (s : Schema) (f : File)  
  (h : gen nm s = .ok f) :  
    ∀ p q, RefEdge s p q → FileRefEdge f (moduleName nm p) (moduleName nm q)
```

## gen\_tree\_sound

Nothing is invented: every edge a reader finds between two tables is one the documents actually induced.

```
theorem gen_tree_sound (nm : Naming) (s : Schema) (f : File)
  (h : gen nm s = .ok f) :
  (∀ t u, t ∈ s → u ∈ s →
    FileCollEdge f (moduleName nm t.path) (moduleName nm u.path) →
    CollEdge s t.path u.path) ∧
  (∀ t u, t ∈ s → u ∈ s →
    FileRefEdge f (moduleName nm t.path) (moduleName nm u.path) →
    RefEdge s t.path u.path)
```

## inj\_of\_nodup\_map

If a list's keys are all distinct then two elements with the same key are the same element.

```
theorem inj_of_nodup_map {α β : Type} {f : α → β} :
  ∀ {l : List α}, (l.map f).Nodup → ∀ {a b : α}, a ∈ l → b ∈ l → f a = f b → a = b
```

## keyTy0f\_id\_not\_ref

A member called `id` that is an object is refused outright, so a reference never hides in the key.

```
theorem keyTy0f_id_not_ref {t : Table} {kt : Ty} {c : Column} {q : Path}
  (h : keyTy0f t = .ok kt) (hn : (t.columns.map Column.name).Nodup)
  (hc : c ∈ t.columns) (hid : c.name = "id") (hty : c.field.ty = .ref q) : False
```

## keyTy0f\_not\_ref

A table's key is never an object.

```
theorem keyTy0f_not_ref {t : Table} {kt : Ty} {p : Path}
  (h : keyTy0f t = .ok kt) : kt ≠ .ref p
```

## layout0f\_keyTy

If the layout was computed, the key's type was settled first.

```
theorem layout0f_keyTy {nm : Naming} {t : Table} {cols : List Col}
  (h : layout0f nm t = .ok cols) : ∃ kt, keyTy0f t = .ok kt
```

## layout0f\_mapM

If the layout was computed, every member was considered.

```
theorem layout0f_mapM {nm : Naming} {t : Table} {cols : List Col}
  (h : layout0f nm t = .ok cols) :
  ∃ ms, t.columns.mapM (memberCol (generatedNames nm t)) = .ok ms
```

## layout0f\_member

A column produced for a member of the table reaches the layout.

```
theorem layout0f_member {nm : Naming} {t : Table} {cols : List Col}
  {c : Column} {col : Col}
  (h : layout0f nm t = .ok cols) (hc : c ∈ t.columns)
  (hmc : memberCol (generatedNames nm t) c = .ok (some col)) : col ∈ cols
```

### mem\_layoutOf

Every member that is neither the table's key nor a collection becomes a column, under its mangled name, at the type inference gave it.

```
theorem mem_layoutOf {nm : Naming} {t : Table} {cols : List Col} {c : Col}
  (h : layoutOf nm t = .ok cols) (hc : c ∈ cols) :
  (∃ kt, keyTyOf t = .ok kt ∧ c ∈ baseCols nm t kt) ∨
  (∃ col, col ∈ t.columns ∧ memberCol (generatedNames nm t) col = .ok (some c))
```

### mem\_orderGo\_complete

And it loses none, given that no two tables share a path.

```
theorem mem_orderGo_complete {t : Table} :
  ∀ (fuel : Nat) (pending acc : List Table),
  (pending.map Table.path).Nodup →
  (t ∈ pending ∨ t ∈ acc) → t ∈ orderGo fuel pending acc
```

### mem\_orderGo\_sound

The compile-order sort invents no tables.

```
theorem mem_orderGo_sound {t : Table} :
  ∀ (fuel : Nat) (pending acc : List Table),
  t ∈ orderGo fuel pending acc → t ∈ pending ∨ t ∈ acc
```

### mem\_orderTables\_complete

The same, said of the sort as a whole.

```
theorem mem_orderTables_complete {t : Table} {s : Schema}
  (hn : (s.map Table.path).Nodup) (h : t ∈ s) : t ∈ orderTables s
```

### mem\_orderTables\_sound

The same, said of the sort as a whole.

```
theorem mem_orderTables_sound {t : Table} {s : Schema} (h : t ∈ orderTables s) : t ∈ s
```

### memberCol\_ref

A member that is an object and is not the key becomes a column holding that object's key.

```
theorem memberCol_ref {gen : List String} {c : Column} {q : Path} {o : Option Col}
  (hok : memberCol gen c = .ok o) (hid : ¬ (c.name == idColumn) = true)
  (hty : c.field.ty = .ref q) :
  o = some { name := mangle c.name, ty := .ref q
    , nullable := c.field.nullable, kind := .member c.name }
```

### memberCol\_ty

A member column keeps the member's own type.

```
theorem memberCol_ty {gen : List String} {c : Column} {col : Col}
  (h : memberCol gen c = .ok (some col)) : col.ty = c.field.ty
```

### noClash\_modules

The duplicate-name check hands back its input unchanged.

```
theorem noClash_modules {mods : List Module} {f : File}
  (h : noClash mods = .ok f) : f.modules = mods
```

---

### nodupPaths\_sound

The check that no two tables share a path means what it says.

```
theorem nodupPaths_sound : ∀ {l : List Path}, nodupPaths l = true → l.Nodup
```

### path\_mem

A path the check found in the schema is the path of a table of it.

```
private theorem path_mem {s : Schema} {p : Path}
  (h : (s.map Table.path).contains p = true) : ∃ u, u ∈ s ∧ u.path = p
```

### reachGo\_sound

So does the whole search.

```
theorem reachGo_sound {s : Schema} : ∀ (fuel : Nat) (acc : List Path),
  (∀ p, p ∈ acc → Reaches s p) → ∀ p, p ∈ reachGo s fuel acc → Reaches s p
```

### reachable\_sound

Everything the search reports is genuinely reachable from the root.

```
theorem reachable_sound {s : Schema} : ∀ p, p ∈ reachable s → Reaches s p
```

### stepReach\_sound

One round of the reachability search only ever reports tables genuinely one edge further out.

```
theorem stepReach_sound {s : Schema} {acc : List Path}
  (hacc : ∀ p, p ∈ acc → Reaches s p) : ∀ p, p ∈ stepReach s acc → Reaches s p
```

## Inference <sup>12</sup>

That the corpus determines the schema, and that each member's type is the tightest one its values allow.

---

### foldJoin\_le

Folding the type-join over a list of observed types lands above every one of them.

```
theorem foldJoin_le : ∀ {l : List Ty} {a τ : Ty},
  l.foldl1 (fun acc t => acc.bind (Ty.join · t)) (some a) = some τ →
  a ⊆ τ ∧ ∀ t, t ∈ l → t ⊆ τ
```

### foldJoin\_least

And lands below anything else that is above them all.

```
theorem foldJoin_least : ∀ {l : List Ty} {a τ σ : Ty},
  l.foldl1 (fun acc t => acc.bind (Ty.join · t)) (some a) = some τ →
  a ⊆ σ → (∀ t, t ∈ l → t ⊆ σ) → τ ⊆ σ
```

### foldJoin\_none

Once the join is undefined it stays undefined, so a fold that succeeded never passed through failure.

```
theorem foldJoin_none (l : List Ty) :
  l.foldl1 (fun acc t => acc.bind (Ty.join · t)) none = none
```

## foldl\_perm\_on

The same, where the laws hold only of values satisfying an invariant.

```
theorem foldl_perm_on {α β : Type} {f : β → α → β} {P : β → Prop} {Q : α → Prop}
  (hP : ∀ b x, P b → Q x → P (f b x))
  (hrc : ∀ b x y, P b → Q x → Q y → f (f b x) y = f (f b y) x) :
  ∀ {l1 l2 : List α}, l1.Perm l2 → (∀ x, x ∈ l1 → Q x) →
    ∀ b, P b → l1.foldl f b = l2.foldl f b
```

## infer\_admits

Every type ever seen at a member fits the type that member was given.

```
theorem infer_admits (cfg : Config) (ds : List Doc) (ts : Tables)
  (h : inferCorpus cfg ds = .ok ts) :
  ∀ p t, (p, t) ∈ ts → ∀ k o, (k, o) ∈ t.members →
    ∀ τ, o.joined = some τ → ∀ ty, ty ∈ o.seen → ty ⊆ τ
```

## infer\_least

And that type is the least one that fits, so nothing is widened further than the data forces.

```
theorem infer_least (cfg : Config) (ds : List Doc) (ts : Tables)
  (h : inferCorpus cfg ds = .ok ts) :
  ∀ p t, (p, t) ∈ ts → ∀ k o, (k, o) ∈ t.members →
    ∀ τ σ, o.joined = some τ → (∀ ty, ty ∈ o.seen → ty ⊆ σ) → τ ⊆ σ
```

## infer\_perm

The schema is a property of the corpus and not of the order its documents arrived in: shuffle the corpus and you get the same answer.

```
theorem infer_perm (cfg : Config) (ds es : List Doc) (h : ds.Perm es)
  {ta tb : Tables} (hda : inferCorpus cfg ds = .ok ta)
  (hdb : inferCorpus cfg es = .ok tb) :
  ta = tb
```

## le\_trans

The type order is transitive.

```
theorem le_trans {a b c : Ty} (hab : a ⊆ b) (hbc : b ⊆ c) : a ⊆ c
```

## mapM\_ok\_of\_all

If every element succeeds then the whole list does.

```
theorem mapM_ok_of_all {α β ε : Type} {f : α → Except ε β} :
  ∀ {l : List α}, (∀ x, x ∈ l → ∃ b, f x = .ok b) → ∃ r, l.mapM f = .ok r
```

## mapM\_perm

Permuted inputs give permuted outputs, when both succeed.

```
theorem mapM_perm {α β ε : Type} {f : α → Except ε β} :
  ∀ {l1 l2 : List α}, l1.Perm l2 → ∀ {r1 r2 : List β},
    l1.mapM f = .ok r1 → l2.mapM f = .ok r2 → r1.Perm r2
```

---

### obs\_admits

Said of one member's observations on their own.

```
theorem obs_admits {o : Obs} {τ : Ty} (h : o.joined = some τ) :  
  ∀ t, t ∈ o.seen → t ⊆ τ
```

### obs\_least

Said of one member's observations on their own.

```
theorem obs_least {o : Obs} {τ σ : Ty} (h : o.joined = some τ)  
  (hσ : ∀ t, t ∈ o.seen → t ⊆ σ) : τ ⊆ σ
```

## Walk <sup>3</sup>

That the traversal which reads the documents keeps its observations in order — the invariant everything in the previous group relies on.

---

### keep

Each record update the walk makes leaves the member list alone.

```
private theorem keep {f : TableObs → TableObs}  
  (hm : ∀ t, (f t).members = t.members) : ∀ t, TableObsOk t → TableObsOk (f t)
```

### observeDocument\_ok

So one document, observed from nothing, comes out in order.

```
theorem observeDocument_ok {cfg : Config} {d : Doc} {ts : Tables}  
  (h : observeDocument cfg d = .ok ts) : TablesOk ts
```

### observe\_ok

Everything the inference walk records is kept in key order -- tables by path, members by name, types by their order.

```
theorem observe_ok (cfg : Config) :  
  ∀ ts target inCollection j, TablesOk ts → ∀ ts',  
    observeObject cfg ts target inCollection j = .ok ts' → TablesOk ts'
```

## Merge <sup>14</sup>

That combining two documents' observations is commutative and associative, at each of the three levels the data has.

---

### Obs.merge\_assoc

And one level down again.

```
theorem Obs.merge_assoc {a b c : Obs} (ha : ObsOk a) (hb : ObsOk b) (hc : ObsOk c) :  
  Obs.merge (Obs.merge a b) c = Obs.merge a (Obs.merge b c)
```

### Obs.merge\_comm

And one level down again, for a single member's: counts add, type sets union, flags disjoint.

```
theorem Obs.merge_comm {a b : Obs} (ha : ObsOk a) (hb : ObsOk b) :  
  Obs.merge a b = Obs.merge b a
```

### Obs.merge\_ok

And one level down again.

```
theorem Obs.merge_ok {a b : Obs} (ha : ObsOk a) (hb : ObsOk b) :  
  ObsOk (Obs.merge a b)
```

### TableObs.merge\_assoc

The same, one level down.

```
theorem TableObs.merge_assoc {a b c : TableObs}  
  (ha : TableObsOk a) (hb : TableObsOk b) (hc : TableObsOk c) :  
  TableObs.merge (TableObs.merge a b) c = TableObs.merge a (TableObs.merge b c)
```

### TableObs.merge\_comm

The same, one level down, for a single table's observations.

```
theorem TableObs.merge_comm {a b : TableObs} (ha : TableObsOk a) (hb : TableObsOk b) :  
  TableObs.merge a b = TableObs.merge b a
```

### TableObs.merge\_ok

The same, one level down.

```
theorem TableObs.merge_ok {a b : TableObs} (ha : TableObsOk a) (hb : TableObsOk b) :  
  TableObsOk (TableObs.merge a b) :=  
  {mergeBy_sorted strOrder _ _ ha.1 hb.1,  
   mergeBy_allV (combine := Obs.merge) (P
```

### TableObsOk.empty

So is a table with no members yet.

```
theorem TableObsOk.empty : TableObsOk ({} : TableObs)
```

### Tables.merge\_assoc

And grouping three of them either way gives the same answer.

```
theorem Tables.merge_assoc {a b c : Tables}  
  (ha : TablesOk a) (hb : TablesOk b) (hc : TablesOk c) :  
  Tables.merge (Tables.merge a b) c = Tables.merge a (Tables.merge b c) :=  
  mergeBy_assocOn (combine := TableObs.merge) (P
```

### Tables.merge\_comm

Merging two documents' observations gives the same answer in either order.

```
theorem Tables.merge_comm {a b : Tables} (ha : TablesOk a) (hb : TablesOk b) :  
  Tables.merge a b = Tables.merge b a :=  
  mergeBy_commOn (combine := TableObs.merge) (P
```

---

### Tables.merge\_ok

Merging preserves the order invariant.

```
theorem Tables.merge_ok {a b : Tables} (ha : TablesOk a) (hb : TablesOk b) :  
  TablesOk (Tables.merge a b) :=  
  {mergeBy_sorted pathOrder _ _ ha.1 hb.1,  
   mergeBy_allV (combine := TableObs.merge) (P
```

---

### Tables.merge\_right\_comm

Together: two documents merged into an accumulator commute, which is what the corpus fold needs.

```
theorem Tables.merge_right_comm {b x y : Tables}  
  (hb : TablesOk b) (hx : TablesOk x) (hy : TablesOk y) :  
  Tables.merge (Tables.merge b x) y = Tables.merge (Tables.merge b y) x
```

---

### Tables.upsert\_ok

Recording a table keeps everything in order.

```
theorem Tables.upsert_ok {ts : Tables} {p : Path} {f : TableObs → TableObs}  
  (h : TablesOk ts) (hf : ∀ t, TableObsOk t → TableObsOk (f t)) :  
  TablesOk (ts.upsert p f)
```

---

### TablesOk.nil

Observing nothing is trivially in order.

```
theorem TablesOk.nil : TablesOk []
```

---

### recordMember\_ok

So does recording one observation at one member.

```
theorem recordMember_ok {ts : Tables} {p : Path} {k : String} {sn : Seen}  
  (h : TablesOk ts) : TablesOk (recordMember ts p k sn)
```

## Order <sub>3</sub>

That the three things used as keys — member names, table paths, types — are each strictly totally ordered.

---

### pathOrder

So are table paths.

```
theorem pathOrder : StrictOrder Path.lt (· == ·) where  
  eq_iff a b
```

---

### strOrder

Member names are strictly totally ordered.

```
theorem strOrder : StrictOrder (fun a b : String => decide (a < b)) (· == ·) where  
  eq_iff a b
```

---

### tyOrder

So are types.

```
theorem tyOrder : StrictOrder Ty.lt (· == ·) where  
  eq_iff a b
```

## Sorted <sup>27</sup>

The generic machinery: sorted association lists, and the fact that merging them is characterised entirely by what they look up.

### allV\_map

And preserves a property of the values.

```
theorem allV_map {P : V → Prop} {g : K × V → K × V} (hg : ∀ p, P p.2 → P (g p).2) :  
  ∀ l : List (K × V), AllV P l → AllV P (l.map g)
```

### belowK\_of\_lt

Transitivity carries `below` down from a smaller key.

```
theorem belowK_of_lt {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) {j k : K} :  
  ∀ {l : List K}, BelowK lt k l → lt j k = true → BelowK lt j l
```

### below\_map

So it leaves `below` alone.

```
theorem below_map {lt : K → K → Bool} {j : K} {g : K × V → K × V}  
  (hg : ∀ p, (g p).1 = p.1) : ∀ l : List (K × V), Below lt j l → Below lt j (l.map g)
```

### below\_of\_lt

The same, for association lists.

```
theorem below_of_lt {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) {j k : K} :  
  ∀ {l : List (K × V)}, Below lt k l → lt j k = true → Below lt j l
```

### except\_bind\_ok

A computation that succeeded had both halves succeed.

```
theorem except_bind_ok {ε α β : Type} {x : Except ε α} {g : α → Except ε β} {b : β}  
  (h : (x >=> g) = .ok b) : ∃ a, x = .ok a ∧ g a = .ok b
```

### insertBy\_allV

And preserves a property of the values.

```
theorem insertBy_allV {lt eqk : K → K → Bool} {combine : V → V → V} {P : V → Prop}  
  (hP : ∀ x y, P x → P y → P (combine x y)) {k : K} {v : V} (hv : P v) :  
  ∀ l : List (K × V), AllV P l → AllV P (insertBy lt eqk combine k v l)
```

### insertBy\_below

And keeps a smaller key below it.

```
theorem insertBy_below {lt eqk : K → K → Bool} {combine : V → V → V} {j k : K} {v : V} :  
  ∀ l : List (K × V), Below lt j l → lt j k = true →  
    Below lt j (insertBy lt eqk combine k v l)
```

### insertBy\_sorted

Inserting one entry keeps the list ordered.

```
theorem insertBy_sorted {lt eqk : K → K → Bool} {combine : V → V → V}  
  (ho : StrictOrder lt eqk) {k : K} {v : V} :  
  ∀ l : List (K × V), SortedBy lt l → SortedBy lt (insertBy lt eqk combine k v l)
```

## lookupBy\_allV

A value looked up satisfies whatever the list's values do.

```
theorem lookupBy_allV {eqk : K → K → Bool} {P : V → Prop} {k : K} {v : V} :  
  ∀ {l : List (K × V)}, AllV P l → lookupBy eqk k l = some v → P v
```

## lookupBy\_of\_below

A key below everything in a list is not in it.

```
theorem lookupBy_of_below {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) {k : K} :  
  ∀ {l : List (K × V)}, Below lt k l → lookupBy eqk k l = none
```

## mapM\_ok\_map

A mapped list agrees pointwise with its inputs under any function that agrees on each.

```
theorem mapM_ok_map {α β γ ε : Type} {f : α → Except ε β} {g : β → γ} {k : α → γ}  
  (hgk : ∀ a b, f a = .ok b → g b = k a) :  
  ∀ {l : List α} {r : List β}, l.mapM f = .ok r → r.map g = l.map k
```

## mapM\_ok\_pointwise

And every input reached some output.

```
theorem mapM_ok_pointwise {α β ε : Type} {f : α → Except ε β} :  
  ∀ {l : List α} {r : List β}, l.mapM f = .ok r →  
    ∀ a, a ∈ l → ∃ b, b ∈ r ∧ f a = .ok b
```

## memBy\_of\_below

A key below everything in a set is not in it.

```
theorem memBy_of_below {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) {k : K} :  
  ∀ {l : List K}, BelowK lt k l → memBy eqk k l = false
```

## mem\_of\_mapM\_ok

Every output of a mapped list came from some input.

```
theorem mem_of_mapM_ok {α β ε : Type} {f : α → Except ε β} :  
  ∀ {l : List α} {r : List β}, l.mapM f = .ok r → ∀ b, b ∈ r → ∃ a, a ∈ l ∧ f a = .ok b
```

## mergeBy\_assocOn

Merging is associative, for the same reason.

```
theorem mergeBy_assocOn {lt eqk : K → K → Bool} {combine : V → V → V} {P : V → Prop}  
  (ho : StrictOrder lt eqk)  
  (hP : ∀ x y, P x → P y → P (combine x y))  
  (hca : ∀ x y z, P x → P y → P z → combine (combine x y) z = combine x (combine y z)) :  
  ∀ a b c : List (K × V), SortedBy lt a → SortedBy lt b → SortedBy lt c →  
    AllV P a → AllV P b → AllV P c →  
    mergeBy lt eqk combine (mergeBy lt eqk combine a b) c  
      = mergeBy lt eqk combine a (mergeBy lt eqk combine b c)
```

## mergeBy\_below

A key below both lists is below their merge.

```
theorem mergeBy_below {lt eqk : K → K → Bool} {combine : V → V → V} {j : K} :  
  ∀ a b : List (K × V), Below lt j a → Below lt j b →  
    Below lt j (mergeBy lt eqk combine a b)
```

## mergeBy\_commOn

Merging is commutative, because combining lookups is.

```
theorem mergeBy_commOn {lt eqk : K → K → Bool} {combine : V → V → V} {P : V → Prop}
  (ho : StrictOrder lt eqk)
  (hc : ∀ x y, P x → P y → combine x y = combine y x) :
  ∀ a b : List (K × V), SortedBy lt a → SortedBy lt b → AllV P a → AllV P b →
    mergeBy lt eqk combine a b = mergeBy lt eqk combine b a
```

## mergeBy\_lookup

A merge is exactly its two lookups combined -- this is the whole content of merging, and both laws below read it off.

```
theorem mergeBy_lookup {lt eqk : K → K → Bool} {combine : V → V → V}
  (ho : StrictOrder lt eqk) :
  ∀ a b : List (K × V), SortedBy lt a → SortedBy lt b → ∀ k,
    lookupBy eqk k (mergeBy lt eqk combine a b)
      = combineOpt combine (lookupBy eqk k a) (lookupBy eqk k b)
```

## mergeBy\_sorted

Merging two ordered lists gives an ordered list.

```
theorem mergeBy_sorted {lt eqk : K → K → Bool} {combine : V → V → V}
  (ho : StrictOrder lt eqk) :
  ∀ a b : List (K × V), SortedBy lt a → SortedBy lt b →
    SortedBy lt (mergeBy lt eqk combine a b)
```

## sortedBy\_map

Rewriting values in place leaves the order alone.

```
theorem sortedBy_map {lt : K → K → Bool} {g : K × V → K × V}
  (hg : ∀ p, (g p).1 = p.1) :
  ∀ l : List (K × V), SortedBy lt l → SortedBy lt (l.map g)
```

## sortedK\_ext

Two sorted sets with the same members are the same list.

```
theorem sortedK_ext {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) :
  ∀ {a b : List K}, SortedK lt a → SortedK lt b →
    (∀ k, memBy eqk k a = memBy eqk k b) → a = b
```

## sorted\_ext

Two sorted lists that agree on every lookup are the same list.

```
theorem sorted_ext {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) :
  ∀ {a b : List (K × V)}, SortedBy lt a → SortedBy lt b →
    (∀ k, lookupBy eqk k a = lookupBy eqk k b) → a = b
```

## unionBy\_assoc

And associative.

```
theorem unionBy_assoc {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) :
  ∀ a b c : List K, SortedK lt a → SortedK lt b → SortedK lt c →
    unionBy lt eqk (unionBy lt eqk a b) c = unionBy lt eqk a (unionBy lt eqk b c)
```

---

### unionBy\_belowK

A key below both sets is below their union.

```
theorem unionBy_belowK {lt eqk : K → K → Bool} {j : K} :  
  ∀ a b : List K, BelowK lt j a → BelowK lt j b → BelowK lt j (unionBy lt eqk a b)
```

### unionBy\_comm

The same for a set: union is commutative.

```
theorem unionBy_comm {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) :  
  ∀ a b, unionBy lt eqk a b = unionBy lt eqk b a
```

### unionBy\_mem

A union is exactly the two memberships, disjointed.

```
theorem unionBy_mem {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) :  
  ∀ a b : List K, SortedK lt a → SortedK lt b → ∀ k,  
    memBy eqk k (unionBy lt eqk a b) = (memBy eqk k a || memBy eqk k b)
```

### unionBy\_sorted

And preserves order.

```
theorem unionBy_sorted {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) :  
  ∀ a b : List K, SortedK lt a → SortedK lt b → SortedK lt (unionBy lt eqk a b)
```

## Lattice <sup>7</sup>

That the type order is a lattice and the join is the least upper bound, which is what makes an inferred type the tightest one.

---

### bot\_le

The empty type sits below everything.

```
theorem bot_le (a : Ty) : Ty.bot ⊆ a
```

### join\_assoc

Nor on how three of them are grouped, which is what makes folding observations order-independent.

```
theorem join_assoc (a b c : Ty) :  
  (Ty.join a b).bind (fun ab => Ty.join ab c)  
  = (Ty.join b c).bind (fun bc => Ty.join a bc)
```

### join\_comm

The common type of two types does not depend on which is named first.

```
theorem join_comm (a b : Ty) : Ty.join a b = Ty.join b a
```

### join\_idem

A type joined with itself is itself.

```
theorem join_idem (a : Ty) : Ty.join a a = some a
```

---

---

### join\_least

The common type is the *\*least\** type above both -- the tightest answer, not merely an answer.

```
theorem join_least {a b c d : Ty} (h : Ty.join a b = some c) (ha : a  $\sqsubseteq$  d) (hb : b  $\sqsubseteq$  d) :  
  c  $\sqsubseteq$  d
```

### le\_join\_left

The common type is above the first argument.

```
theorem le_join_left {a b c : Ty} (h : Ty.join a b = some c) : a  $\sqsubseteq$  c
```

### le\_join\_right

And above the second.

```
theorem le_join_right {a b c : Ty} (h : Ty.join a b = some c) : b  $\sqsubseteq$  c
```

## Wellformed <sup>4</sup>

That the emitted signature never repeats a name.

---

### certify\_wellFormed

The generator checks its own output before returning it.

```
theorem certify_wellFormed {f g : File} (h : certify f = .ok g) : g.WellFormed
```

### gen\_wellFormed

Every generated signature is well formed: no module, value or record field name repeats.

```
theorem gen_wellFormed (nm : Naming) (s : Schema) (f : File) (h : gen nm s = .ok f) :  
  f.WellFormed
```

### nodupNames\_sound

And so does its distinctness test.

```
theorem nodupNames_sound :  $\forall$  {l : List String}, nodupNames l = true  $\rightarrow$  l.Nodup
```

### okB\_sound

That check means what the proposition says.

```
theorem okB_sound {f : File} (h : f.okB = true) : f.WellFormed
```

05

## Proved, but not needed for the guarantee

A further 52 results are proved in the repository and do not appear in the tree above. That is not an oversight, and in one case it is the point.

**Name mangling is the interesting case.** One might expect “distinct members become distinct columns” to be what makes the signature well formed. It is not how the proof goes: `gen` checks its own output for repeated names before returning it, and `gen_wellFormed` is proved by inverting that check. So well-formedness holds of *every* schema, including ones inference could never build, and it does not rest on `mangle_injective` at all. Injectivity is what makes the design sound rather than lucky — without it the

generator would be relying on a runtime check to catch a collision it should never create — but the theorem above stands without it.

The rest are results that were superseded. `toSchema_paths_nodup` and its neighbours proved distinctness from the walk invariant; once `gen` began checking the same condition itself, the shorter route won. They are kept because they say something true about inference that the check does not.

## Tree <sup>2</sup>

- `gen_edges_only_on_tables` — Only a table's own module carries edges; loaders and the registry carry none.
- `parentOfElement_shorter` — Following a table to its parent strictly shortens the path, so the tree has no cycles and the walk up from any table terminates.

## Inference <sup>1</sup>

- `foldl_perm` — Folding an operation that commutes in this sense over a list does not depend on the order.

## Walk <sup>2</sup>

- `inferCorpus_ok` — Everything inference builds is kept in key order.
- `inferFinish_ok` — The final pass, which fills in absence counts, keeps it that way.

## Merge <sup>3</sup>

- `toSchema_mem` — A table of the schema came from an observation of that path.
- `toSchema_members_nodup` — No table has two columns of one name.
- `toSchema_paths_nodup` — No two tables of the schema share a path.

## Order <sup>1</sup>

- `segOrder` — Path segments are strictly totally ordered.

## Sorted <sup>14</sup>

- `StrictOrder.eq_symm` — Key equality does not depend on which side is named first.
- `StrictOrder.irrefl` — Nothing is strictly below itself -- which follows from asymmetry rather than being assumed.
- `allV_mem` — A value in a list satisfies whatever all its values do.
- `below_not_mem` — A key below everything in a list does not appear in it.
- `combineOpt_assoc` — Nor on how three are grouped.
- `combineOpt_comm` — Combining two lookups does not depend on their order.
- `insertSorted_perm` — Inserting one entry into a sorted list permutes it.
- `lookupBy_head` — The head of a sorted list looks up to its own value.
- `mergeBy_allV` — Merging preserves a property of the values.
- `mergeBy_assoc` — Merging is associative, for combining functions with no side conditions.

- `mergeBy_comm` — Merging is commutative, proved directly by induction rather than through lookups.
- `mergeBy_right_comm` — Two merges into an accumulator commute.
- `sortBy_perm` — So insertion sort returns a permutation of its input.
- `sortedBy_nodup` — A list sorted by a strict order repeats no key.

## Mangle <sup>29</sup>

- `encodeByte_append_inj` — Because it is prefix-free: one encoded byte determines where it ends.
- `encodeByte_ne_nil` — An encoded byte is never empty.
- `encodeBytes_eq_nil` — Only the empty name encodes to nothing.
- `encodeBytes_injective` — The byte encoding is injective.
- `encodeBytes_ne_f2` — No encoded name can begin with the escape prefix.
- `encodeBytes_ne_snoc_underscore` — Nor can it end in an underscore.
- `encodeBytes_ne_underscore2` — Nor can it be a lone pair of underscores.
- `hexDigit_inj` — Hex digits are distinct.
- `hexDigit_ne_underscore` — And none of them is an underscore.
- `hexVal_hexDigit` — Reading a hex digit back gives what was written.
- `isKeyword_props` — What it means to be a keyword, as the proof uses it.
- `keywordSuffixed_inj` — Distinct keywords stay distinct once escaped.
- `keywords_len` — Every OCaml keyword is at least two characters.
- `lt_128_valid` — Every byte below 128 is a valid character.
- `mangleChars_of_mangle` — The mangled name is the encoding of the name's characters.
- `mangle_injective` — Two distinct member names never give one OCaml identifier, so two members never become one column and two tables never one module.
- `mangle_starts_lower` — A mangled name begins with a lowercase letter, which is what makes capitalising one character injective.
- `no_underscore_in_keywords` — No keyword contains an underscore.
- `pass_ne_underscore` — A character that passes through unescaped is not the escape character.
- `passesThrough_lt` — Characters that pass through are exactly the safe ones.
- `prefixedCore_inj` — Prefixed and unprefixed names stay disjoint.
- `prefixedCore_ne_snoc` — A prefixed name is never a keyword with an underscore appended.
- `prefixedCore_starts_lower` — The escaped form begins with a lowercase letter too.
- `startsLower_cons` — Prefixing a lowercase letter keeps a name starting lowercase.
- `toNat_inj` — Character codes are distinct for distinct characters.
- `toNat_lt_256` — An encoded byte is a byte.
- `toNat_ofNat` — Round-tripping a byte through a character is the identity.
- `toNat_ofNat_pass` — The same, for characters that pass through unescaped.

- `toUTF8_data_toList_inj` — Distinct strings have distinct UTF-8 encodings.
- 

06

## What is not proved

No goal in the repository is left open, so this section is about scope rather than about unfinished work.

1. **That inference never trips the generator's check.** Guarantees 3–6 hold because `gen` checks its input has the shape a corpus produces and refuses one that does not. That is the stronger arrangement for a reader of the output — the properties hold of every file emitted, whatever it was handed — but it does not by itself say the check never fires on a real corpus. It does not: tables are keyed by path, a repeated member is refused, and the walk records a table before descending into what it holds. Turning that sentence into a theorem is the obvious next piece.
2. **That the emitted OCaml compiles.** Well-formedness constrains names, not references. A module naming a unit that is compiled after it satisfies the definition and still does not build. The generator does emit units in a topological order; nothing yet states that it is one.
3. **Anything about rows surviving.** This program emits a description of tables and never emits a row, so there is no object for a preservation theorem to be about. A round trip could only ever hold up to member order, explicit `null` versus an absent key, and the written form of a number — all three discarded by design.